DINI VUMIJOJO

PET 361 602037

Contents

Introduction 2
Website Setup 2
Database Creation 2
Client data 3
Database Connection 3
Normal Login Functionality 4
SQL Injection Simulation 5
Cross-Site Scripting (XSS) Simulation 5
Code Explanation & Comments 6
Login credentials 7
final code used with comments as explanation 7
Conclusion 14
References 15

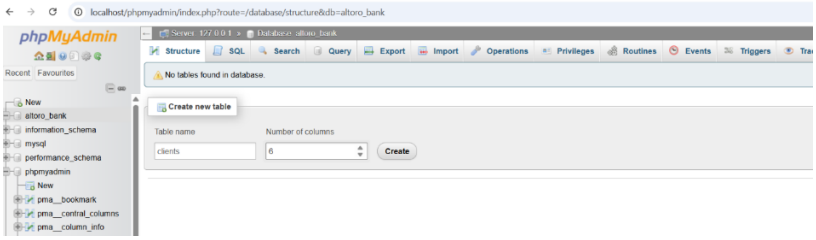
Introduction

- This project focuses on the simulation of common web application vulnerabilities, specifically SQL Injection and Cross-Site Scripting (XSS), using an offline replica of the AltoroMutual banking login page. The purpose is to demonstrate how insecure coding practices can lead to serious data exposure risks.
- The system was developed using HTML, CSS, PHP, and MySQL within a local environment powered by XAMPP. This ensures that all testing is performed safely without affecting any real-world systems.
- The project aims to provide a practical understanding of how attackers exploit vulnerabilities, while also highlighting the importance of secure authentication mechanisms and proper input validation in modern web applications.

Website Setup

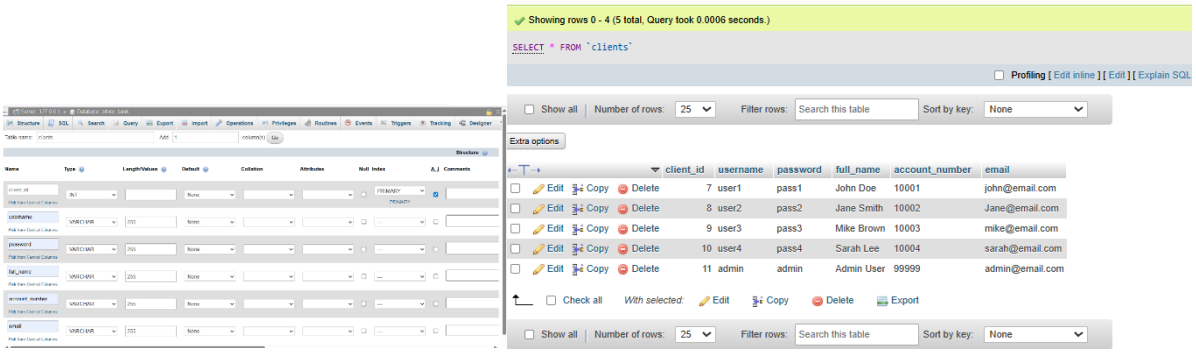
- The AltoroMutual login page was recreated locally using semantic HTML and CSS to closely match the layout, structure, and design of the original website.
- The page includes key components such as the navigation bar, sidebar menu, login form, and footer, ensuring that the interface resembles a real banking application.
- The website was hosted locally using XAMPP, allowing it to run on http://localhost without being deployed online, in compliance with project requirements.
- Only the login page functionality was implemented, while other links and navigation elements were included for visual accuracy but do not perform real actions.
- The use of semantic HTML elements such as <header>, <nav>, <main>, and <section> improves the structure and readability of the code.

Database Creation



- A MySQL database named **altoro_bank** was created using phpMyAdmin to store client information securely.
- A table called **clients** was created within the database to hold user data required for authentication and simulation purposes.
- The table consists of six fields: client_id, username, password, full_name, account_number, and email, ensuring compliance with the requirement of having between 5 and 10 fields.
- The **client_id** field was set as the primary key and configured to auto-increment, ensuring each record is uniquely identified.
- Data types were carefully selected, with VARCHAR used for textual fields and INT used for the client_id, ensuring proper storage and retrieval of information.
-

Client data



Five client records were inserted into the database to simulate real users interacting with the system.

Each client contains login credentials (username and password) as well as personal information such as full name, account number, and email address.

The usernames and passwords were intentionally kept simple to ensure ease of testing and marking, as required by the project instructions.

The data was inserted using phpMyAdmin's insert functionality, allowing for manual control over each record.

Database Connection

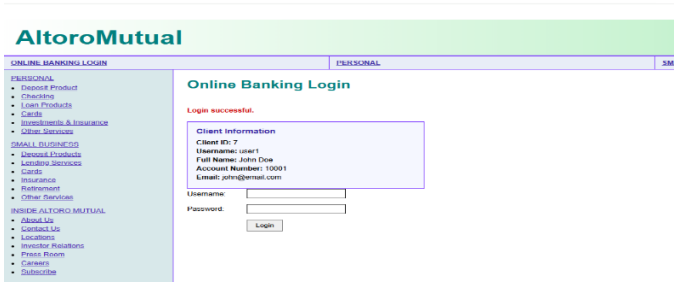
```
$conn = new mysqli("localhost", "root", "", "altoro_bank");

if ($conn->connect_error) {
```

```
  die("Connection failed: " . $conn->connect_error);
}
```

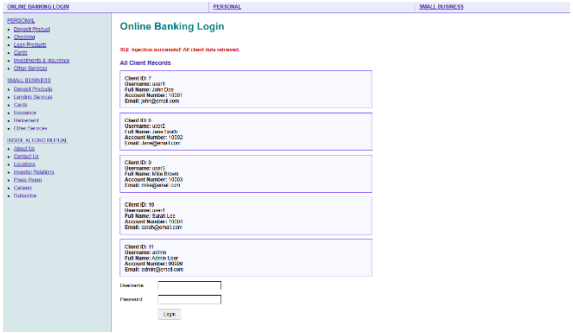
- The website was connected to the MySQL database using PHP and the mysqli extension, allowing communication between the frontend and backend.
- A connection was established using the localhost server, with the default root user and no password, as configured by XAMPP.
- Error handling was implemented to ensure that any connection issues are detected and displayed, preventing silent failures.
- The connection enables the system to execute SQL queries for authentication and data retrieval.
- This setup simulates how real-world web applications interact with backend databases to validate users and manage sensitive information.

Normal Login Functionality



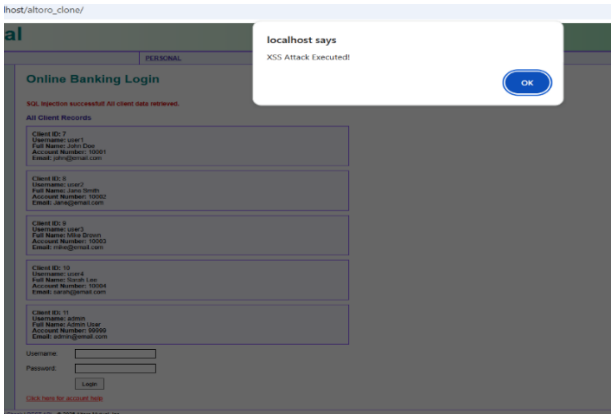
- A login system was implemented using a form that collects a username and password from the user.
- When the form is submitted, PHP processes the input and constructs an SQL query to check whether the credentials match any record in the database.
- If a match is found, the system retrieves the corresponding client's data and displays it on the page.
- This demonstrates a typical authentication workflow used in many web applications.
- If the credentials are incorrect, an error message is displayed indicating invalid login details.
- The system ensures that only one user's data is displayed during a valid login attempt.
- This functionality confirms that the database connection and query execution are working correctly.
- The login process forms the baseline against which the SQL injection simulation is compared.

SQL Injection Simulation



- SQL Injection was simulated by entering a malicious payload (' OR '1'='1) into the login form.
- This payload represents a common attack technique used to manipulate SQL queries and bypass authentication.
- The system was programmed to detect the presence of suspicious input (such as single quotes) and trigger a simulation of an injection attack.
- Instead of executing the normal login query, the system retrieves all client records from the database.
- All user data is then displayed on the page, demonstrating how an attacker could gain unauthorized access to sensitive information.
- A clear message is displayed indicating that an SQL Injection attack has been successfully simulated.
- This simulation highlights the dangers of unsanitized input and the importance of secure query handling in web applications.

Cross-Site Scripting (XSS) Simulation



- Cross-Site Scripting (XSS) was simulated using a clickable link that executes a JavaScript function.

- When the link is clicked, a script runs in the browser and displays an alert message.
- This demonstrates how malicious scripts can be executed within a web application if input is not properly validated or sanitized.
- The simulation shows how attackers can inject scripts into web pages to manipulate user interactions or steal sensitive information.
- The use of JavaScript highlights how client-side vulnerabilities can be exploited independently of the server.
- The XSS simulation provides a clear visual representation of how such attacks behave in real-world scenarios.
- This reinforces the importance of input validation and output encoding in preventing XSS vulnerabilities.

Code Explanation & Comments

SQL Injection logic:

```php
if (strpos($username, "'") !== false || strpos($password, "'") !== false) {
    // SQL Injection simulation
    $sql = "SELECT * FROM clients";
```

Normal login:

```php
// Normal login
$sql = "SELECT * FROM clients WHERE username='$username' AND password='$password'";
```

XSS:

```html
<a href="#" onclick="showXSS()">Click here for account help</a>
```

```javascript
function showXSS() {
    alert('XSS Attack Executed!');
}
```

- Comments were added throughout the code to clearly identify sections responsible for SQL Injection and XSS simulations.
- The SQL Injection logic includes a conditional statement that checks for suspicious input and modifies query behavior accordingly.
- The normal login logic is separated from the attack simulation, ensuring clarity in how each process operates.
- The XSS simulation is implemented using a JavaScript function that is triggered by a user interaction.
- These comments improve code readability and make it easier to understand the purpose and functionality of each section.

Login credentials

The following login credentials can be used to access the system:

- user1 / pass1
- user2 / pass2
- user3 / pass3
- user4 / pass4
- admin / admin

final code used with comments as explanation

```
<?php
$conn = new mysqli("localhost", "root", "", "altoro_bank");
if ($conn->connect_error) {
  die("Connection failed: " . $conn->connect_error);
}
$message = "";
$userData = null;
$allUsers = [];

if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $username = $_POST["username"];
  $password = $_POST["password"];

  if (strpos($username, "' OR") !== false || strpos($password, "' OR") !== false) {
    // SQL Injection simulation  detects malicious input and returns all client records
    $sql = "SELECT * FROM clients";
    $result = $conn->query($sql);

    if ($result) {
```

```
        $allUsers = $result->fetch_all(MYSQLI_ASSOC);
        $message = "SQL Injection successful! All client data retrieved.";
      }
    } else {
      // Normal login: checks username and password against database
      $sql = "SELECT * FROM clients WHERE username='$username' AND
password='$password'";
      $result = $conn->query($sql);


      if ($result && $result->num_rows === 1) {
        $userData = $result->fetch_assoc();
        $message = "Login successful.";
      } else {
        $message = "Invalid username or password.";
      }
    }
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
```

```
<!-- Top Links -->
<header>
<nav class="top-links">
      <a href="#">Sign In</a> |
      <a href="#">Contact Us</a> |
      <a href="#" id="xssLink">Feedback</a> |
      <span>Search</span>
      <input type="text" class="search-box">
      <button>Go</button>
   </nav>

   <!-- Logo & Banner -->
   <section class="header">
      <h1 class="logo">AltoroMutual</h1>
      <p class="banner">DEMO SITE ONLY</p>
   </section>

   <!-- Navigation Bar -->
   <nav class="nav-bar">
      <ul>
         <li>ONLINE BANKING LOGIN</li>
         <li>PERSONAL</li>
         <li>SMALL BUSINESS</li>
         <li>INSIDE ALTORO MUTUAL</li>
      </ul>
   </nav>
</header>
```

```
<main class="main-container">

  <!-- Sidebar -->
  <aside class="sidebar">
    <section>
      <h4>PERSONAL</h4>
      <ul>
        <li><a href="#">Deposit Product</a></li>
        <li><a href="#">Checking</a></li>
        <li><a href="#">Loan Products</a></li>
        <li><a href="#">Cards</a></li>
        <li><a href="#">Investments & Insurance</a></li>
        <li><a href="#">Other Services</a></li>
      </ul>
    </section>

    <section>
      <h4>SMALL BUSINESS</h4>
      <ul>
        <li><a href="#">Deposit Products</a></li>
        <li><a href="#">Lending Services</a></li>
        <li><a href="#">Cards</a></li>
        <li><a href="#">Insurance</a></li>
        <li><a href="#">Retirement</a></li>
        <li><a href="#">Other Services</a></li>
      </ul>
    </section>
```

```
        <section>
          <h4>INSIDE ALTORO MUTUAL</h4>
          <ul>
            <li><a href="#">About Us</a></li>
            <li><a href="#">Contact Us</a></li>
            <li><a href="#">Locations</a></li>
            <li><a href="#">Investor Relations</a></li>
            <li><a href="#">Press Room</a></li>
            <li><a href="#">Careers</a></li>
            <li><a href="#">Subscribe</a></li>
          </ul>
        </section>
      </aside>

      <!-- Main Content -->
      <section class="content">
        <h2>Online Banking Login</h2>
        <?php if (!empty($message)): ?>
   <p class="message"><?php echo $message; ?></p>
<?php endif; ?>

<?php if ($userData): ?>
   <section class="user-info">
      <h3>Client Information</h3>
      <p><strong>Client ID:</strong> <?php echo $userData['client_id']; ?></p>
      <p><strong>Username:</strong> <?php echo $userData['username']; ?></p>
      <p><strong>Full Name:</strong> <?php echo $userData['full_name']; ?></p>
```

```
      <p><strong>Account Number:</strong> <?php echo $userData['account_number']; ?></p>

      <p><strong>Email:</strong> <?php echo $userData['email']; ?></p>

    </section>

<?php endif; ?>


<?php if (!empty($allUsers)): ?>

    <section class="all-users">

      <h3>All Client Records</h3>

<!-- XSS Simulation: clicking this link triggers a JavaScript alert -->

      <?php foreach ($allUsers as $user): ?>

        <article class="user-card">

          <p><strong>Client ID:</strong> <?php echo $user['client_id']; ?></p>

          <p><strong>Username:</strong> <?php echo $user['username']; ?></p>

          <p><strong>Full Name:</strong> <?php echo $user['full_name']; ?></p>

          <p><strong>Account Number:</strong> <?php echo $user['account_number']; ?></p>

          <p><strong>Email:</strong> <?php echo $user['email']; ?></p>

        </article>

      <?php endforeach; ?>

    </section>

<?php endif; ?>


        <form action="" method="POST">

          <div class="form-row">

            <label for="username">Username:</label>

            <input type="text" id="username" name="username" required>

          </div>
```

```
            <div class="form-row">
              <label for="password">Password:</label>
              <input type="password" id="password" name="password" required>
            </div>

            <div class="form-row">
              <button type="submit" name="login">Login</button>
            </div>
          </form>
          <p>
   <a href="#" onclick="showXSS()">Click here for account help</a>
</p>

<script>
function showXSS() {
   alert('XSS Attack Executed!');
}
</script>
      </section>
  </main>
  <!-- Footer -->
   <footer class="footer">
      <nav>
         <a href="#">Privacy Policy</a> |
         <a href="#">Security Statement</a> |
         <a href="#">Server Status Check</a> |
         <a href="#">REST API</a>
      </nav>
```

```
    <p>© 2026 Altoro Mutual, Inc.</p>
  </footer>


  <!-- Disclaimer -->
  <section class="disclaimer">
    <p>
      This website is for educational purposes only to demonstrate web application vulnerabilities.
      This is not a real banking site.
    </p>
  </section>
 <script src="script.js"></script>
</body>
</html>
```

Conclusion

- This project successfully demonstrates how web application vulnerabilities such as SQL Injection and Cross-Site Scripting can compromise sensitive data and system integrity.
- It emphasizes the importance of secure coding practices, input validation, and proper database interaction to prevent such attacks in real-world applications.

References

OWASP, 2023. SQL Injection. Available at: https://owasp.org [Accessed 8 April 2026].

OWASP, 2023. Cross-Site Scripting (XSS). Available at: https://owasp.org [Accessed 8 April 2026].

PHP, 2023. MySQLi Documentation. Available at: https://www.php.net [Accessed 8 April 2026].